React + Vite React Router v6 TailwindCSS v4 Super Admin & ANO views

Admin Portal *Frontend Guide*

This guide covers every screen, component, state rule, and design decision for the two admin roles — Super Admin (Unit HQ) and College Admin (ANO). Build from top to bottom. Every section includes a visual mockup, component spec, and implementation notes.

Color tokens

All colors are defined as CSS custom properties on `:root`. Never hardcode hex values in components — always reference a token. This makes light/dark mode trivial to add later.

```
/* globals.css */
:root {
  /* Backgrounds */
  --bg-base:     #f7f6f2;   /* page */
  --bg-surface:  #ffffff;   /* cards, panels */
  --bg-muted:    #f0ede8;   /* hover rows, inputs */
  --bg-sidebar:  #0e0f11;   /* left nav */

  /* Text */
  --text-primary:   #0e0f11;
  --text-secondary: #3a3d45;
  --text-muted:     #7a7f8e;
  --text-inverse:   #f0ede6;

  /* Brand */
  --brand-navy:  #1a3a5c;   /* primary action */
  --brand-red:   #c8461e;   /* destructive / alert */
  --brand-green: #2d7a4f;   /* success / active */
  --brand-gold:  #b8860b;   /* NCC accent */

  /* Borders */
  --border:       #d5d2ca;
  --border-focus: #1a3a5c;

  /* Semantics */
  --status-draft:     #888;
  --status-published: #1a3a5c;
  --status-closed:    #c8461e;
  --status-active:    #2d7a4f;
}
```

Typography

Two fonts. **Geist** for all UI text (labels, tables, body). **Instrument Serif** for display headings only (page titles, dashboard greeting). DM Mono for all code, status badges, and monospaced metadata.

```
/* index.html */
<link href="https://fonts.googleapis.com/css2?family=DM+Mono:wght@400;500&family=Instrument+Serif:ital@0;1&fami

/* tailwind.config.js */
theme: {
  extend: {
    fontFamily: {
      sans:  ['Geist', 'sans-serif'],
      serif: ['Instrument Serif', 'serif'],
      mono:  ['DM Mono', 'monospace'],
    }
  }
}
```

TYPE SCALE

TOKEN	USAGE	SIZE / WEIGHT
display	Page title, dashboard greeting	32–40px / Instrument Serif
heading	Section headings within a page	20px / Geist 600
subheading	Card titles, panel labels	15px / Geist 500
body	Descriptions, help text	14px / Geist 400
label	Form labels, table headers	11px / Geist 600 / uppercase / tracked
mono	Badges, IDs, code, regimental nos	11–12px / DM Mono

Components

BUTTON VARIANTS

```
// Button.tsx
type Variant = 'primary' | 'ghost' | 'danger' | 'outline'

// primary   → bg-[--brand-navy] text-white
// ghost     → bg-transparent text-[--text-secondary] hover:bg-[--bg-muted]
// danger    → bg-[--brand-red]/10 text-[--brand-red] border border-[--brand-red]/30
// outline   → border border-[--border] text-[--text-primary] hover:border-[--brand-navy]

const base = 'h-9 px-4 rounded-md text-sm font-medium inline-flex items-center gap-2 transition-all duration-15
```

STATUS BADGE

Used in exam lists, user tables, attempt tracking. Always rendered in DM Mono.

```
// StatusBadge.tsx
const variants = {
  DRAFT:     'bg-gray-100 text-gray-500',
  PUBLISHED: 'bg-[--brand-navy]/10 text-[--brand-navy]',
  CLOSED:    'bg-[--brand-red]/10 text-[--brand-red]',
  ACTIVE:    'bg-[--brand-green]/10 text-[--brand-green]',
}
// class="font-mono text-[10px] px-2 py-0.5 rounded-full"
```

TABLE

All admin tables follow the same pattern: sticky header, hover rows, action column pinned right. Never paginate client-side — use server-side pagination with limit/offset and pass `totalCount` from the API.

```
// AdminTable.tsx — core structure
<div className="overflow-x-auto rounded-lg border border-[--border]">
  <table className="w-full text-sm">
    <thead className="bg-[--bg-muted] sticky top-0">
      <tr>
        <th className="font-mono text-[10px] uppercase tracking-widest text-[--text-muted] px-4 py-3 text-left'
          {column.header}
        </th>
      </tr>
    </thead>
    <tbody>
      <tr className="border-t border-[--border] hover:bg-[--bg-muted]/50 transition-colors">
        <td className="px-4 py-3">...</td>
      </tr>
    </tbody>
  </table>
</div>
```

FORM FIELDS

```
// All input fields follow this pattern
<div className="flex flex-col gap-1.5">
  <label className="font-mono text-[10px] uppercase tracking-widest text-[--text-muted]">
    {label}
  </label>
  <input
    className="h-9 px-3 rounded-md border border-[--border] bg-[--bg-surface]
               text-sm text-[--text-primary] placeholder:text-[--text-muted]
               focus:outline-none focus:ring-2 focus:ring-[--brand-navy]/20
               focus:border-[--border-focus] transition-all"
  />
</div>
```

STAT CARD

```
// StatCard.tsx
<div className="bg-[--bg-surface] border border-[--border] rounded-lg p-5">
  <p className="font-mono text-[10px] uppercase tracking-widest text-[--text-muted] mb-2">
    {label}
  </p>
  <p className="text-3xl font-semibold text-[--text-primary] tabular-nums">
    {value}
  </p>
  <p className="text-xs text-[--text-muted] mt-1">{subtext}</p>
</div>
```

Shell layout

The admin shell is a fixed sidebar + scrollable content area. The sidebar is icon-only at small widths (56px), expanding to 220px with labels on `lg:` screens. On mobile it collapses to a hamburger-triggered drawer.

```
// AdminShell.tsx
<div className="flex h-screen bg-[--bg-base]">
  <Sidebar />
  <div className="flex flex-col flex-1 overflow-hidden">
    <Topbar />
    <main className="flex-1 overflow-y-auto p-6 lg:p-8">
      <Outlet />   {/* React Router */}
    </main>
  </div>
</div>
```

SIDEBAR NAV ITEMS

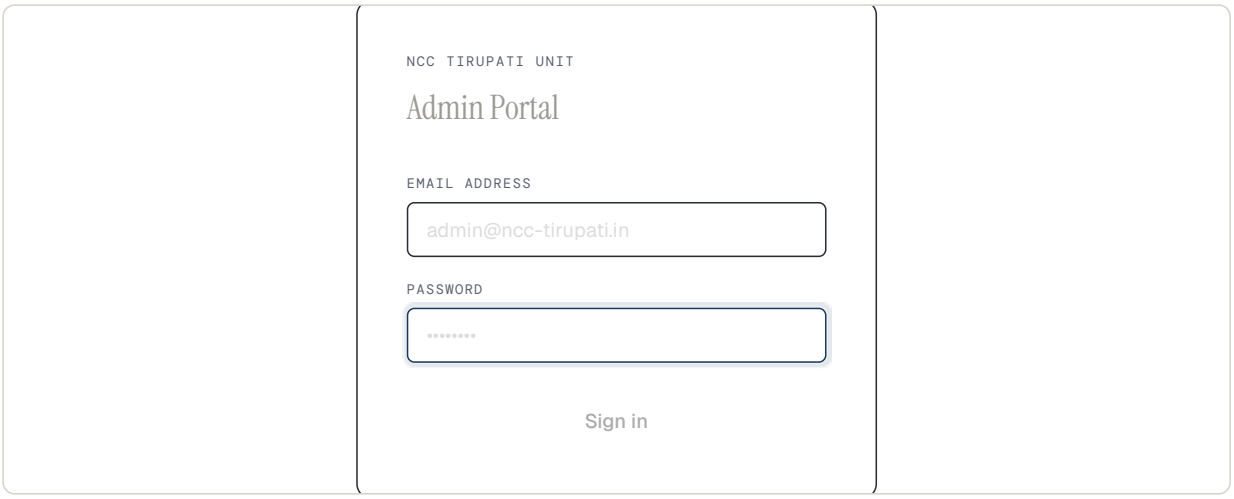
ICON	LABEL	ROLE
⊞	Dashboard	Both
👥	Users	Both
📋	Exams	Both
🏛	Colleges	Super Admin only
📊	Results	Both
⚙	Settings	Both

> **ROLE VISIBILITY**
> Hide the Colleges nav item entirely for ANO role — don't just disable it. ANOs should never know that global college management exists. Derive visibility from the `role` field decoded from the JWT payload stored in auth context.

Screen: Login

Two separate login forms rendered conditionally via a toggle — **Cadet login** (Regimental No + Password) and **Admin login** (Email + Password). The admin portal URL is `/admin/login`, so only the admin form should appear here. No toggle needed on this route.

LOGIN SCREEN — ADMIN



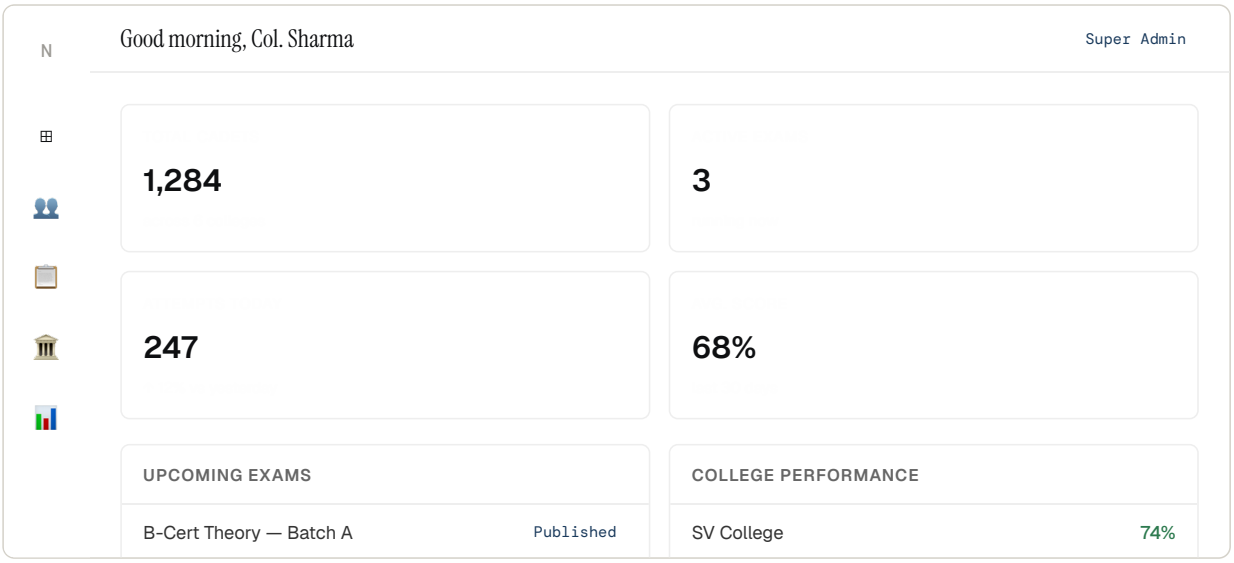
Implementation notes

- On submit: `POST /api/auth/login` with `{ email, password, role: 'ADMIN' }`
- Store `accessToken` in React context (memory only — not localStorage)
- The `refreshToken` arrives as an `httpOnly` cookie — no JS access needed
- On 401: show inline error message below the form, never an alert/toast
- On 403 (disabled account): show a distinct "Your account is disabled" message
- After 5 failed attempts: disable the submit button for 60 seconds with a countdown

Screen: Dashboard

The first screen after login. Shows key stats, upcoming exams, and recent activity. Content differs by role — Super Admin sees unit-wide data, ANO sees college-scoped data.

DASHBOARD — SUPER ADMIN VIEW



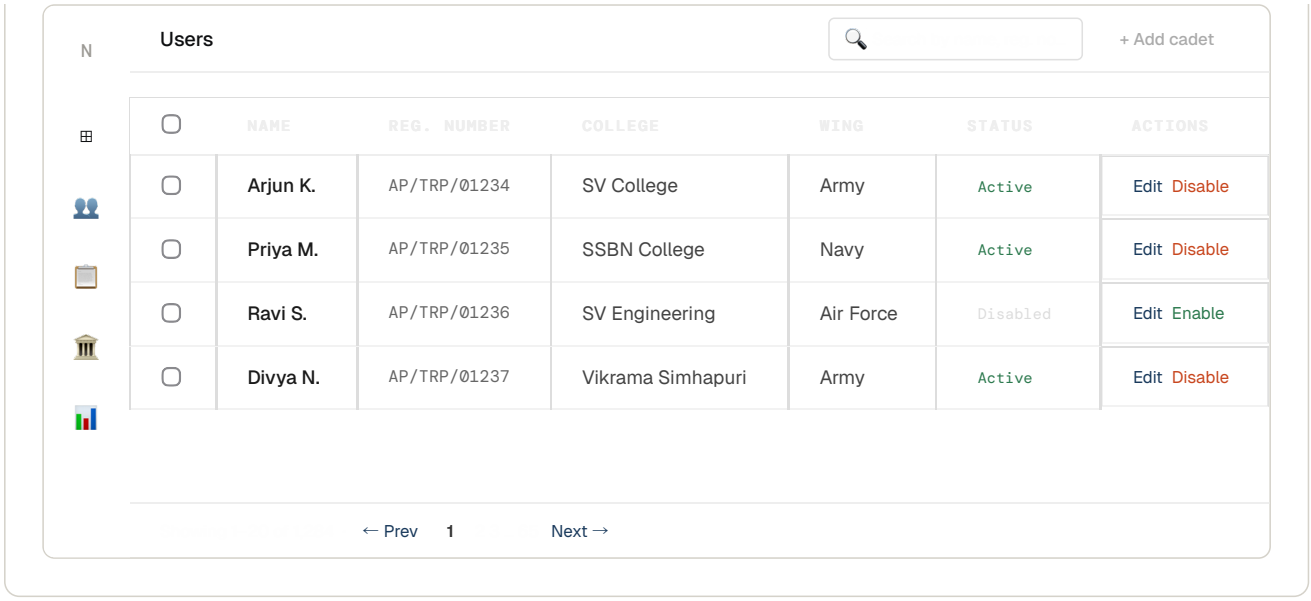
Stat cards to show

CARD	SUPER ADMIN	ANO (COLLEGE ADMIN)
Cadets	Unit-wide total	College total only
Active exams	Global + college	College-scoped
Attempts today	Unit-wide	College-scoped
Avg. score	Unit-wide, 30d	College-scoped, 30d

Screen: User management

The most-used admin screen. Lists all cadets with search, filter by college/wing/status, and bulk actions. ANO can only see and manage cadets from their own college.

USER MANAGEMENT TABLE



Filters to implement

- **Search:** name or regimental number — debounced 300ms, hits `GET /api/users?q=...`
- **College:** dropdown, Super Admin only. ANO sees no filter (always their college).
- **Wing:** Army / Navy / Air Force multi-select
- **Status:** Active / Disabled toggle

Bulk actions (appear when ≥1 row is checked)

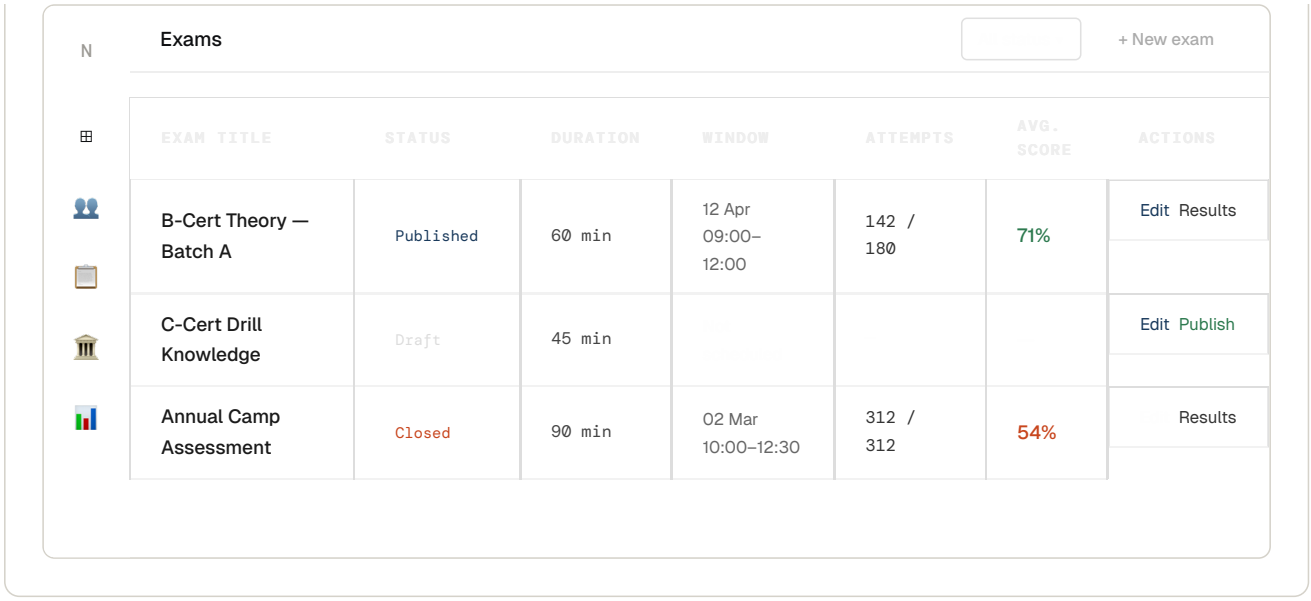
- Bulk enable / disable
- Bulk assign to college (Super Admin only)
- Export selected as CSV

Add/Edit cadet — slide-over panel

Editing opens a right-side drawer (not a modal). Width: 440px. Fields: Name, Regimental No, Email (optional), Phone (optional), Wing, College, Password (create only). Never show the current password — only allow reset.

Screen: Exam list

Lists all exams with their status, attempt count, and schedule. Officers create exams from here; Super Admin can see all, ANO sees only their college's exams.

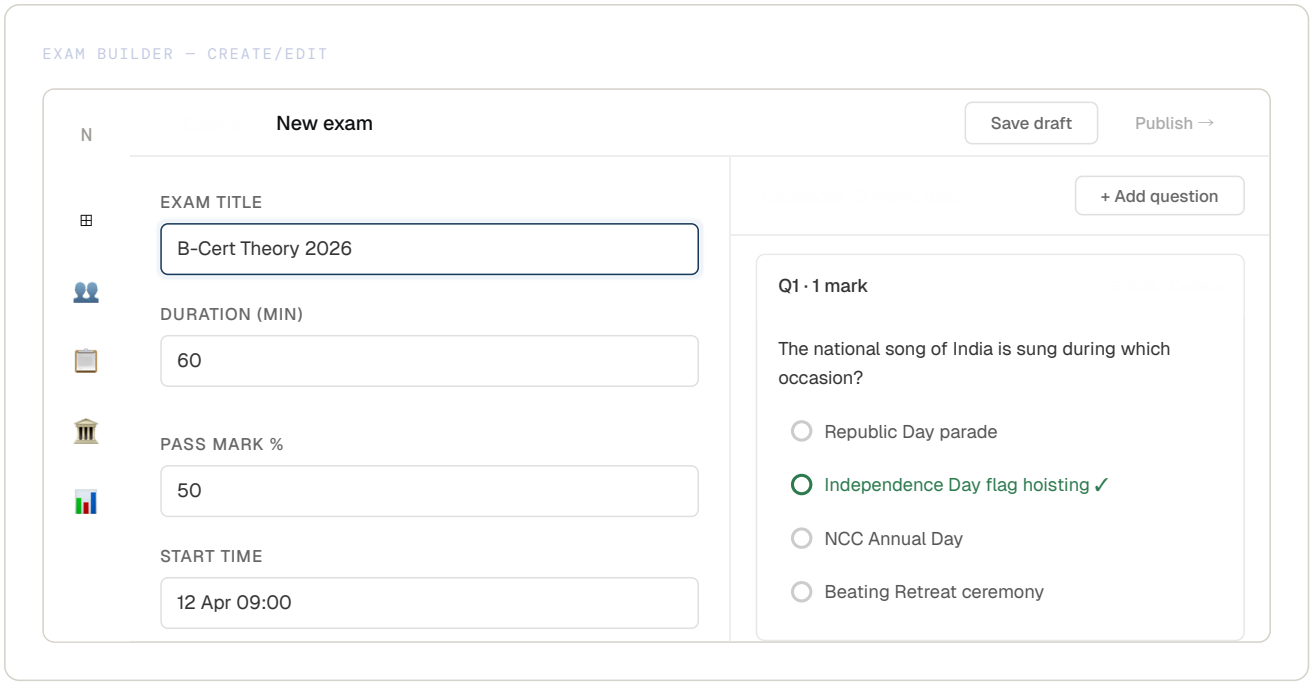


PUBLISH GUARD

The "Publish" action must run a client-side pre-flight check before calling the API: (1) exam has at least 1 question, (2) every question has exactly 1 correct option marked, (3) start time is in the future. Show a checklist modal listing any failed conditions — never silently skip.

Screen: Exam builder

Two-panel layout. Left panel: exam metadata. Right panel: question list + builder. Questions are added and reordered within the page — no navigation away.



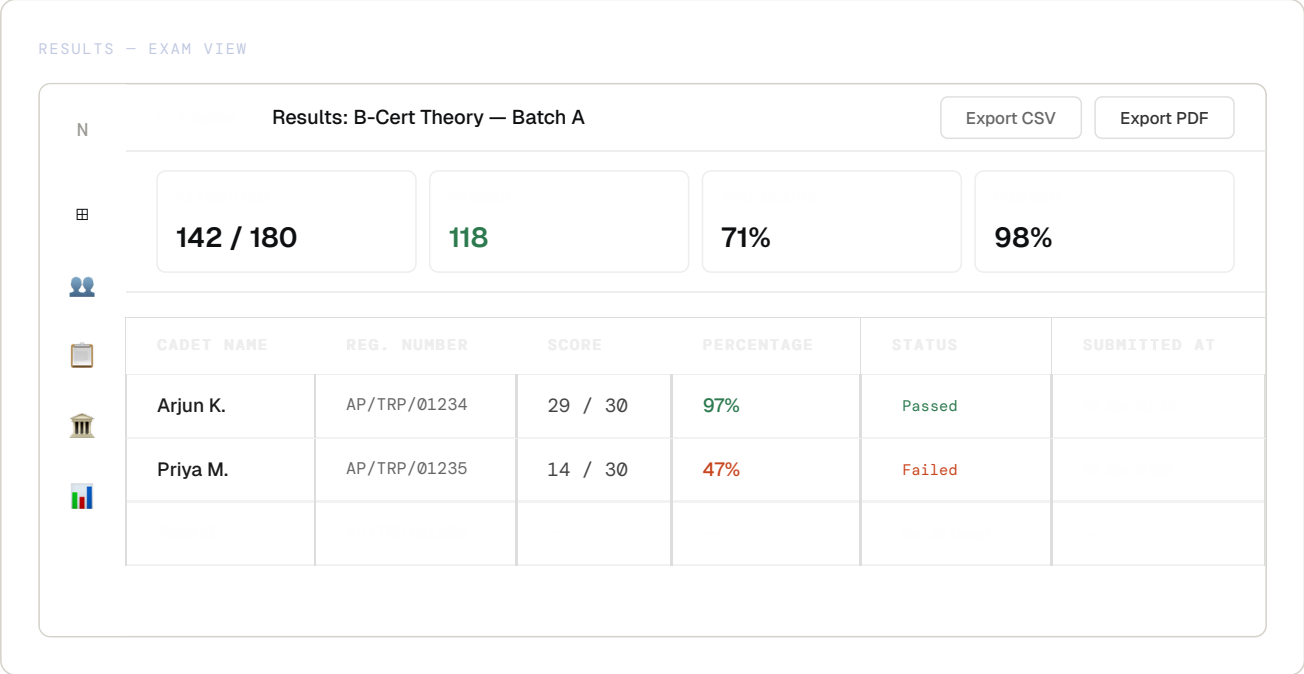
Question builder — inline form

When "+ Add question" is clicked, an inline card appears at the bottom of the question list (not a modal). Fields: question text (textarea), marks (number, default 1), 4 option fields, one radio to mark the correct answer. Save button appends it to the list. No page navigation.

> **BULK IMPORT**
> Add a "Import CSV" button in the question panel header. Accept a CSV with columns: `question, option_a, option_b, option_c, option_d, correct_option` . Validate client-side before uploading. This will save ANOs enormous time during initial setup.

Screen: Results

Accessible from the exam list ("Results" action) or the sidebar. Shows per-attempt scores in a table with filters. Export to CSV/PDF.



Result visibility rules

- Super Admin: sees all results across all colleges
- ANO: sees only results for cadets in their college
- Individual cadet result page: accessible at `/admin/results/:attemptId` — shows per-question breakdown (which answer the cadet picked, which was correct)

Screen: College management

Super Admin only. Lists all affiliated colleges with cadet count and assigned ANO. Simple CRUD — add, edit, deactivate. No complex sub-screens.

Fields per college

- **Name** — full college name
- **Code** — short identifier used in regimental numbers (e.g. `TRP`)
- **City**
- **ANO email** — the College Admin assigned to this college
- **Status** — Active / Inactive

> **NOTE**
> Deactivating a college should NOT delete cadets or exams. It should hide the college from dropdowns and prevent new logins for cadets of that college. Use a `status` field, not a hard delete.

Route map

```
// App.tsx — React Router v6 structure
<Routes>
  <Route path="/admin/login" element={<LoginPage />} />

  {/* Protected: both admin roles */}
  <Route element={<AdminGuard />}>
    <Route element={<AdminShell />}>
      <Route path="/admin"              element={<Dashboard />} />
      <Route path="/admin/users"         element={<UserList />} />
      <Route path="/admin/exams"         element={<ExamList />} />
      <Route path="/admin/exams/new"     element={<ExamBuilder />} />
      <Route path="/admin/exams/:id/edit" element={<ExamBuilder />} />
      <Route path="/admin/exams/:id/results" element={<ExamResults />} />
      <Route path="/admin/results/:attemptId" element={<AttemptDetail />} />

      {/* Super Admin only */}
      <Route element={<SuperAdminGuard />}>
        <Route path="/admin/colleges" element={<CollegeList />} />
      </Route>
    </Route>
  </Route>
</Routes>
```

PATH	COMPONENT	ACCESS
`/admin/login`	LoginPage	Public

/admin	Dashboard	Both roles
/admin/users	UserList	Both roles
/admin/exams	ExamList	Both roles
/admin/exams/new	ExamBuilder	Both roles
/admin/exams/:id/edit	ExamBuilder	Owner or Super Admin
/admin/exams/:id/results	ExamResults	Both roles
/admin/results/:attemptId	AttemptDetail	Both roles
/admin/colleges	CollegeList	Super Admin only

State management

Use React Context + `useReducer` for auth state. Use TanStack Query (React Query) for all server state. Do not put API data into Context — that's what React Query's cache is for.

```
// AuthContext.tsx
type AuthState = {
  user: {
    id: string
    name: string
    email: string
    role: 'SUPER_ADMIN' | 'COLLEGE_ADMIN'
    collegeId: string | null  // null for Super Admin
  } | null
  accessToken: string | null
}

// Never store accessToken in localStorage.
// Store in useState/useReducer inside AuthProvider.
// On page refresh: call POST /api/auth/refresh (uses httpOnly cookie).
// If refresh fails → user is null → redirect to /admin/login.
```

AUTH STATE INITIALIZATION ON PAGE LOAD

```
// AuthProvider.tsx
useEffect(() => {
  // On mount, silently refresh to restore session
  refreshAccessToken()
    .then(token => dispatch({ type: 'SET_TOKEN', token }))
    .catch(() => dispatch({ type: 'LOGOUT' }))
}, [])
```

REACT QUERY — KEY CONVENTIONS

```
// Query key factory — keeps cache keys consistent
export const keys = {
  users:     (filters) => ['users', filters],
  user:      (id)      => ['users', id],
  exams:     (filters) => ['exams', filters],
  exam:      (id)      => ['exams', id],
  results:   (examId)  => ['results', examId],
  colleges:  ()        => ['colleges'],
}

// Mutation: after creating/updating an exam, invalidate the list
const mutation = useMutation({
  mutationFn: createExam,
  onSuccess: () => queryClient.invalidateQueries({ queryKey: keys.exams({}) })
})
```

API layer

Centralize all API calls in an `api/` folder. Use a single Axios instance with request/response interceptors. Never call `fetch` or `axios` directly from components.

```
// api/client.ts
import axios from 'axios'

export const client = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  withCredentials: true,  // sends httpOnly refresh cookie
})

// Attach access token to every request
client.interceptors.request.use(config => {
  const token = getAccessToken()   // from AuthContext
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})

// On 401: attempt silent refresh, then retry original request once
client.interceptors.response.use(
  res => res,
  async err => {
    const original = err.config
    if (err.response?.status === 401 && !original._retry) {
      original._retry = true
      const token = await refreshAccessToken()
      original.headers.Authorization = `Bearer ${token}`
      return client(original)
    }
    return Promise.reject(err)
  }
)
```

```
// api/exams.ts — example module
export const getExams = (filters) =>
  client.get('/exams', { params: filters }).then(r => r.data)

export const createExam = (data) =>
  client.post('/exams', data).then(r => r.data)

export const publishExam = (id) =>
  client.patch(`/exams/${id}/publish`).then(r => r.data)

export const getExamResults = (examId) =>
  client.get(`/exams/${examId}/results`).then(r => r.data)
```

Route guards

```
// guards/AdminGuard.tsx
export function AdminGuard() {
  const { user } = useAuth()
  if (!user) return <Navigate to="/admin/login" replace />
  if (user.role !== 'SUPER_ADMIN' && user.role !== 'COLLEGE_ADMIN')
    return <Navigate to="/admin/login" replace />
  return <Outlet />
}

// guards/SuperAdminGuard.tsx
export function SuperAdminGuard() {
  const { user } = useAuth()
  if (user?.role !== 'SUPER_ADMIN')
    return <Navigate to="/admin" replace />
  return <Outlet />
}
```

> **GUARDS ARE UI-ONLY**
> Route guards prevent navigation — they do not replace server-side authorization. Every API endpoint must independently verify the role from the JWT. A user who manipulates the client JS can bypass a guard; they cannot bypass the backend.

Rules & conventions

FILE STRUCTURE

```
src/
  api/
```

```
    client.ts
    auth.ts
    users.ts
    exams.ts
    results.ts
    colleges.ts
  components/
    ui/           # Button, Badge, Input, Table, Drawer, etc.
    admin/        # AdminShell, Sidebar, Topbar, StatCard
  context/
    AuthContext.tsx
  guards/
    AdminGuard.tsx
    SuperAdminGuard.tsx
  hooks/
    useAuth.ts
    useExams.ts
    useUsers.ts
  pages/
    admin/
      LoginPage.tsx
      Dashboard.tsx
      UserList.tsx
      ExamList.tsx
      ExamBuilder.tsx
      ExamResults.tsx
      AttemptDetail.tsx
      CollegeList.tsx
  utils/
    dates.ts      # format helpers
    validators.ts # exam publish pre-flight
```

HARD RULES

- **Never compute scores client-side.** The score displayed always comes from the API response — never calculate it from the answers array in the frontend.
- **Never store tokens in localStorage.** Access token lives in React state. Refresh token lives in httpOnly cookie only.
- **All destructive actions need a confirm step.** Disable user, close exam, delete question — all require a confirmation dialog. Use a reusable `<ConfirmDialog />` component.
- **Tables are server-paginated.** Never load all records into the browser. Pass `page` and `limit` as query params. Default limit: 20.
- **Form validation runs before API calls.** Use react-hook-form + Zod. Never submit a form with empty required fields.
- **ANO data is always scoped to their `collegeId`.** The frontend passes no `collegeId` filter — the backend injects it from the JWT. This means ANOs cannot spoof another college's data even if they edit request params.
- **Error responses must surface to the user.** Never silently swallow API errors. Show the `message` from the response body inline near the relevant action, not just a generic toast.

PERFORMANCE CONSTRAINTS

- Route-based code splitting with `React.lazy()` — each page is a separate chunk
- Table rows are virtualized if count > 200 (use TanStack Virtual)
- Images (profile photos etc.) use lazy loading with `loading="lazy"`
- Debounce all search inputs by 300ms before triggering API calls

ACCESSIBILITY

- All interactive elements are keyboard navigable (Tab + Enter/Space)
- Status badges include `aria-label` with the full status text
- Modals and drawers trap focus and close on Escape
- Form errors are linked to their input via `aria-describedby`